

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных технологий
Кафедра теоретической кибернетики и прикладной математики

**Разработка мобильной части информационной системы управления
мероприятиями**
выпускная квалификационная работа

Выполнил:
студент группы 4.201-2,
Золотухина Анастасия Максимовна

(подпись)

Научный руководитель:
к.ф.-м.н., доцент
Журавлёва Вера Владимировна

(подпись)

Работа защищена:
«__» _____ 2026 г.

Оценка: _____

Председатель ГЭК:
д.т.н., профессор
Фамилия Имя Отчество

(подпись)

Допустить к защите:
Зав. кафедрой, к.ф.-м.н., доцент
Понькина Елена Владимировна

(подпись)

«__» _____ 2026 г.

РЕФЕРАТ

Тема выпускной квалификационной работы: «Разработка мобильной части информационной системы управления мероприятиями».

Цель работы – разработка мобильной части информационной системы управления мероприятиями.

Объект исследования – методология и практика разработки клиент-серверных приложений.

Предмет исследования – мобильная разработка на языке Kotlin.

В ходе выполнения работы были решены следующие задачи: изучены основные методы и технологии мобильной разработки, выбраны архитектурные принципы и паттерны проектирования, выбраны программные средства разработки, разработан доменный слой, реализовано взаимодействие с backend-частью, разработан пользовательский интерфейс, разработана рекомендательная система.

Итогом исследования стала клиентская часть системы «Events App», которая дает пользователям возможность проходить авторизацию, знакомиться со списком доступных мероприятий, регистрировать свои события, видеть персонализированные предложения и передавать информацию на сервер.

Ключевые слова: разработка мобильных приложений, Kotlin, Android-разработка, фреймворк Jetpack Compose, паттерн MVVM, библиотека Hilt, система управления событиями, рекомендательная система, архитектура клиент-сервер, интерфейс приложения, интеграция с backend-частью.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ МОБИЛЬНОЙ РАЗРАБОТКИ	8
1.1. Концепция мобильного приложения	8
1.2. Обзор платформы Android и выбор языка Kotlin	14
1.3. Архитектурные принципы и паттерны в мобильной разработке	18
1.4. Методы создания UI и работы с данными в Android	23
2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	28
2.1. Анализ требований к системе	28
2.2. Архитектура системы и моделирование компонентов	28
2.3. Моделирование данных на мобильной стороне	28
2.4. Проектирование рекомендательной системы	28
3. РАЗРАБОТКА МОБИЛЬНОЙ-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	29
3.1. Разработка доменного слоя	29
3.2. Взаимодействие с backend-частью	29
3.3. Проектирование пользовательского интерфейса	29
3.4. Интеграция рекомендательной системы	29
3.5. Тестирование и оптимизация приложения	29
ЗАКЛЮЧЕНИЕ	30
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	31
ПРИЛОЖЕНИЕ	35

ВВЕДЕНИЕ

Актуальность исследования. Мероприятия различного вида проводятся ежедневно в миллионах организаций по всему миру. Организаторам приходится решать множество задач: от выбора подходящего места и формата до распространения анонсов в мессенджерах и социальных сетях. В крупных компаниях, где события организуются регулярно, и организаторы, и участники сталкиваются с проблемами координации: ручное ведение списков регистраций в таблицах или отслеживание изменений в расписании приводят к ошибкам, потере времени и снижению эффективности работы.

Для решения этих проблем предлагается разработать информационную систему управления мероприятиями. Система будет встроена в существующую экосистему организации и позволит контролировать весь процесс в одном месте. Она обеспечит централизованный контроль всех этапов: от создания события с указанием даты, описания и места до управления регистрацией участников и анализа вовлеченности. Мобильная часть предоставит удобный доступ к основным функциям: просмотр списка событий, аутентификация пользователей и синхронизация данных с сервером.

Актуальность темы обусловлена растущей цифровизацией всех отраслей экономики, что влечёт за собой увеличение потребности рынка в новых цифровых решениях для привычных и рутинных задач.

Степень разработанности проблемы. В настоящее время существует ряд систем управления мероприятиями: Eventbrite, Google Calendar или корпоративные инструменты такие как Microsoft Teams. Они решают задачи планирования и регистрации, однако зачастую ориентированы на крупные

события и не учитывают специфику внутренних корпоративных процессов. Например, персонализированные рекомендации и полноценная интеграция с мобильными устройствами на базе Android. Автоматизация мобильной части для таких систем остается недостаточно проработанной, особенно в контексте использования Kotlin и современных подходов к разработке. Это создает пробел в доступных решениях для средних организаций.

Разработка предлагаемой информационной системы требует применения современных подходов и технологий: использование паттернов проектирования, фреймворков, технологий тестирования и баз данных.

Целью данной выпускной квалификационной работы является разработка мобильной части информационной системы управления мероприятиями.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Изучить основные методы и технологии мобильной разработки.
2. Выбрать архитектурные принципы и паттерны проектирования.
3. Выбрать программные средства разработки.
4. Разработать доменный слой.
5. Разработать взаимодействие с backend-частью.
6. Разработать пользовательский интерфейс.
7. Разработать систему интеллектуального поиска и рекомендаций.

Объектом исследования данной работы является методология и практика разработки клиент-серверных приложений.

Предмет исследования – мобильная разработка на языке Kotlin.

Теоретическая и методологическая основа исследования включает работы отечественных и зарубежных практиков и теоретиков в области мобильной разработки: документацию Android и Kotlin, а также исследования по паттернам проектирования (MVVM) и алгоритмам обработки текста (TF-IDF). Основу составляют технологии Jetpack Compose, Hilt, а также Retrofit для интеграции с backend. Для управления состоянием приложения и асинхронной загрузки данных применяется Coroutines совместно с Material UI.

Методы исследования: анализ научной литературы, системный анализ, сравнение технологий мобильной разработки, моделирование архитектуры, программирование на Kotlin, тестирование интерфейса и интеграции.

Отличительными особенностями разработки является интеграция системы интеллектуального поиска на основе алгоритма TF-IDF. Данный подход позволяет разбивать поисковые запросы на составляющие компоненты и находить наиболее близкие по тематике мероприятия даже при неполном совпадении, что обеспечит более гибкий поиск по сравнению с традиционными методами. На основе этой же технологии реализована система персонализированных рекомендаций, которая учитывает предпочтения пользователей. Также применяется Jetpack Compose для создания интерфейса, что обеспечивает более плавную и быструю работу по сравнению с традиционными подходами на основе XML. Это повышает удобство для корпоративных пользователей и снижает нагрузку на устройство.

Практическая значимость разработки мобильной части системы управления мероприятиями заключается в том, чтобы предоставить удобный и визуально привлекательный интерфейс для работы пользователей. Благодаря системе организация сможет уменьшить время на координацию, повысить

интерес сотрудников с помощью персонализированных рекомендаций и снизить количество ошибок при регистрации. В результате это улучшит внутренние процессы и повысит общую эффективность.

Структура работы обусловлена последовательностью решения поставленных задач и состоит из введения, трех глав, заключения, библиографического списка и приложений. Общий объем работы составляет + страниц, работа содержит + рисунков, + таблиц и 1 приложение.

В первой главе рассматриваются теоретические основы мобильной разработки: концепция мобильного приложения, обзор платформы Android и выбор языка Kotlin, инструменты разработки, архитектурные принципы и паттерны, методы создания интерфейса и анализ альтернативных подходов.

Вторая глава посвящена проектированию информационной системы управления мероприятиями: анализ требований, архитектура и моделирование компонентов, моделирование данных на мобильной стороне и проектирование рекомендательной системы.(демо)

В третьей главе представлена разработка мобильной части системы: доменный слой, взаимодействие с backend, проектирование пользовательского интерфейса, интеграция рекомендательной системы, тестирование и оптимизация приложения.

Таким образом, проект представляет собой не только создание мобильного приложения, а комплексное решение для оптимизации корпоративных процессов. Оно интегрирует современные технологии разработки, поисковые и рекомендательные механизмы, заменяя рутинные операции эффективным и удобным инструментом для пользователей.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ МОБИЛЬНОЙ РАЗРАБОТКИ

1.1. Концепция мобильного приложения

Мобильные приложения стали неотъемлемой частью повседневной жизни, изменив способ взаимодействия людей с цифровыми технологиями. Согласно данным отчета Digital 2025, пользователи проводят в среднем 5,1 часа в день за мобильными устройствами, из которых 88% времени приходится на приложения. В бизнес-среде такие приложения служат инструментом маркетинга и вовлечения аудитории: компании персонализируют коммуникации через push-уведомления и интеграцию с социальными сетями, что повышает вовлеченность пользователей и оптимизирует рабочие процессы.

Приложение для мобильных устройств – это специально созданный программный продукт, предназначенный для функционирования на портативной технике: смартфонах, планшетах и других устройствах схожего типа. Такое ПО является автономной программой, адаптированной под особенности той или иной операционной системы, что гарантирует эффективное применение ресурсов устройства. В отличие от приложений для ПК и ноутбуков, мобильные версии рассчитаны на переносимость, учитывают условия эксплуатации и обеспечивают комфортный интерфейс взаимодействия с пользователем. Благодаря этому они способны решать множество задач – от получения данных и обмена сообщениями до выполнения рабочих обязанностей, становясь незаменимым средством цифровизации ежедневной жизни.

По технологии разработки мобильные приложения подразделяются на три основных типа.

Нативные приложения разрабатываются исключительно для определенной операционной системы (Android, iOS и других) с использованием специализированных инструментов и языков программирования данной платформы. Такие приложения характеризуются высокой скоростью работы, полноценным доступом к аппаратным возможностям устройства, эффективным расходованием ресурсов аккумулятора и оперативной памяти, а также способностью работать в автономном режиме без подключения к сети. Основным недостатком является необходимость создания отдельного программного кода для каждой платформы, что значительно увеличивает время и финансовые затраты на разработку.

Веб-приложения представляют собой адаптированные версии интернет-сайтов, доступ к которым осуществляется непосредственно через браузер мобильного устройства. Данный тип приложений не требует установки и функционирует на любом устройстве, оснащенном браузером. К преимуществам относятся универсальность применения на различных платформах и упрощенный процесс обновления. Недостатками являются ограниченные возможности взаимодействия с аппаратными компонентами устройства, необходимость постоянного интернет-соединения и сниженная производительность в сравнении с нативными решениями.

Гибридные приложения объединяют преимущества нативных и веб-приложений. В их основе лежат веб-технологии (HTML, CSS, JavaScript), которые интегрируются в специальную нативную оболочку. Такой подход

позволяет размещать приложения в цифровых магазинах Google Play и App Store, применять единую кодовую базу для различных платформ и сокращать расходы на разработку. К недостаткам можно отнести несколько меньшую скорость работы по сравнению с нативными приложениями и возможное несоответствие визуального оформления стандартам отдельных операционных систем.

Параметр	Нативные	Веб-приложения	Гибридные
Производительность	Высокая	Низкая	Средняя
Доступ к функциям устройства	Полный	Ограниченный	Частичный
Работа офлайн	Да	Нет	Ограниченная
Кроссплатформенность	Нет	Да	Да
Время разработки	Долгое	Быстрое	Среднее
Стоимость разработки	Высокая	Низкая	Средняя
Размещение в магазинах	Да	Нет	Да
Обновление	Через магазин	На сервере	Через магазин
Расход ресурсов	Низкий	Высокий	Средний
Примеры	WhatsApp, Instagram	Twitter Lite, Alibaba	Uber, Evernote

Таблица 1. Сравнительная характеристика типов мобильных приложений

Android представляет собой мобильную операционную систему, созданную на основе модифицированного ядра Linux и разработанную

компанией Google. Являясь открытой платформой с доступным исходным кодом, Android предоставляет производителям устройств возможность свободно адаптировать систему под свои требования, внедряя собственные графические оболочки. Данная особенность обеспечивает широкое распространение системы: она поддерживает устройства различных ценовых категорий, от бюджетных до премиум моделей, и предлагает обширную экосистему приложений через магазин Google Play. Однако открытость платформы создает проблемы фрагментации системы и повышенные риски в области информационной безопасности.

iOS является операционной системой для мобильных устройств, разработанной компанией Apple. В отличие от Android, iOS представляет собой закрытую проприетарную платформу, функционирующую исключительно на устройствах Apple. Данный подход обеспечивает высокий уровень интеграции аппаратных и программных компонентов, строгий контроль качества приложений через магазин App Store и повышенную информационную безопасность. Система отличается плавностью пользовательского интерфейса и оптимизацией энергопотребления. К недостаткам относятся высокая стоимость устройств и ограниченные возможности настройки системы.

HarmonyOS представляет собой операционную систему, созданную компанией Huawei в качестве альтернативы Android. Это универсальное решение предназначено для устройств Huawei и ориентировано на технологию распределенных вычислений, обеспечивающую бесшовное взаимодействие между различными устройствами. Система построена на архитектуре микроядра, поддерживает запуск приложений Android и предлагает собственные сервисы HMS. К преимуществам относятся высокий

уровень безопасности и расширенная поддержка технологий, объединяющих устройства в единую систему. Недостатками являются ограниченная доступность приложений за пределами азиатского региона и зависимость от устройств производства Huawei.

Параметр	Android	iOS	HarmonyOS
Тип платформы	Открытая	Закрытая	Закрытая
Доля рынка (2025)	70%	25-30%	4-5%
Количество пользователей	>3 млрд	1 млрд	500 млн
Настройка системы	Высокая	Ограниченная	Средняя
Ценовой сегмент	Все категории	Премиум	Средний-премиум
Экосистема приложений	Google Play	App Store	HMS
Безопасность	Средняя	Высокая	Высокая
Интеграция устройств	Средняя	Высокая	Очень высокая
Фрагментация	Высокая	Низкая	Низкая
Энергоэффективность	Средняя	Высокая	Высокая
География использования	Высокая	Высокая	Ограниченная

Таблица 2. Сравнительная характеристика мобильных операционных систем

Согласно данным на 2025 год Android занимает около 70% мирового рынка мобильных операционных систем. Эта доминирующая позиция обусловлена открытостью платформы, широким выбором устройств

различных ценовых категорий и развитой экосистемой приложений в магазине Google Play. Количество активных пользователей Android превышает 3 миллиарда человек, что делает платформу привлекательной для разработчиков.

В рамках данной работы была выбрана разработка нативного мобильного приложения для платформы Android на языке Kotlin. Выбор данного подхода связан с необходимостью обеспечить высокую производительность, полный доступ к API платформе и создать пользовательский интерфейс, соответствующий стандартам Material Design. Система управления мероприятиями будет решать задачи координации процессов – от регистрации участников до формирования персонализированных рекомендаций, интегрируясь в корпоративную среду и уменьшая вероятность ошибок при создании событий.

1.2. Обзор платформы Android и выбор языка Kotlin

Операционная система Android занимает ведущую позицию на рынке мобильных платформ. В данном разделе будет рассмотрена история становления платформы, особенности её архитектуры, а также объясняется выбор языка программирования Kotlin и интегрированной среды разработки IntelliJ IDEA для реализации проекта.

Разработка Android началась с 2003 года в компании Android Inc, основанной Энди Рубином. Изначально проект был ориентирован на создание программного обеспечения для цифровых камер. В 2005 году корпорация Google приобрела компанию и перевела разработку на создание операционной системы для мобильных устройств. Первый официальный выпуск Android 1.0 состоялся в сентябре 2008 года одновременно с релизом смартфона HTC Dream. За годы платформа прошла существенный путь развития, включающий расширение функциональных возможностей, повышение производительности и усиление системы безопасности. На сегодняшний день Android представляет собой операционную систему, интегрирующую технологии искусственного интеллекта для персонализации пользовательского интерфейса, обеспечивающую защиту конфиденциальных данных посредством современных методов шифрования и управления разрешениями, а также поддерживающую широкий спектр устройств.

Архитектура Android организована по многоуровневому принципу. В основе системы находится модифицированное ядро Linux, обеспечивающее управление памятью, процессами, сетевыми соединениями и безопасностью. Следующий уровень представляет собой слой аппаратной абстракции,

который позволяет производителям адаптировать систему под различные типы оборудования без внесения изменений. Среда исполнения Android Runtime выполняет компиляцию программного кода в машинные инструкции, что обеспечивает высокую производительность приложений. Верхний уровень составляет фреймворк приложений, включающий готовые компоненты для построения пользовательских интерфейсов, фоновых служб и механизмов взаимодействия между программами. Такая организация обеспечивает гибкость и масштабируемость системы.

Для разработки приложений под Android применяются два основных языка программирования – Java и Kotlin. Java использовалась с момента создания платформы и обладает обширной базой библиотек и инструментов. Kotlin же был представлен компанией JetBrains в 2011 году как современная альтернатива, сохраняющая полную совместимость с Java.

Параметр	Java	Kotlin
Объем кода	Большой	Компактный
Null-безопасность	Ручная	Автоматическая
Асинхронность	Сложная	Простая (корутины)
Скорость разработки	Медленнее	Быстрее
Количество ошибок	Больше	Меньше
Поддержка Google	Стандартная	Приоритетная
Совместимость	Только Java	Java + Kotlin

Таблица 3. Сравнительная характеристика языков Java и Kotlin

Kotlin обладает рядом существенных преимуществ по сравнению с Java. Встроенная система защиты от null-указателей предотвращает

возникновение `NullPointerException` – одной из наиболее частых причин сбоев в приложениях. Данный механизм обеспечивает проверку на этапе компиляции, что исключает обращение к несуществующим объектам в процессе выполнения программы. Механизм функций-расширений позволяет добавлять новые методы к существующим классам без изменения их исходного кода. Корутины представляют собой инструмент для упрощения асинхронного программирования, позволяя писать последовательный код для операций, выполняемых параллельно, что избавляет от необходимости использования сложных конструкций обратного вызова. Автоматическое приведение типов после проверки условий избавляет от необходимости явных преобразований, что уменьшает дублирование кода.

В мае 2017 года на конференции Google I/O компания Google официально объявила Kotlin основным языком для разработки приложений под Android. С 2019 года Kotlin стал самым используемым языком в среде разработки Android Studio, что свидетельствует о признании его преимуществ профессиональным сообществом.

Выбор Kotlin для реализации данного проекта обусловлен несколькими факторами. Система управления мероприятиями требует надежной обработки пользовательских данных, где механизм защиты от отсутствия значений играет главную роль в обеспечении стабильности приложения. Корутины существенно упрощают реализацию операций, выполняемых в фоновом режиме, включая обмен данными с сервером и работу с локальной базой данных. Компактность синтаксиса ускоряет разработку и повышает читаемость кода, что важно для последующего сопровождения и расширения функциональности. Совместимость с библиотеками Java обеспечивает доступ

к обширной экосистеме инструментов. Официальная поддержка Google и активное сообщество разработчиков гарантируют актуальность и развитие языка.

В качестве интегрированной среды разработки была выбрана IntelliJ IDEA Ultimate Edition производства компании JetBrains. Учитывая, что Kotlin и IntelliJ IDEA разработаны одной компанией, среда обеспечивает наиболее качественную поддержку данного языка программирования. IntelliJ IDEA предлагает развитые средства рефакторинга кода, гибкую систему плагинов, улучшенную интеграцию с системами контроля версий и поддержку множества языков программирования. Это обеспечивает универсальность инструмента, что особенно важно для проектов, предполагающих возможное включение серверных компонентов или интеграцию с другими платформами.

1.3. Архитектурные принципы и паттерны в мобильной разработке

Архитектура мобильного приложения представляет собой организационную структуру программного кода, которая определяет взаимодействие компонентов системы и распределение ответственности между ними. Правильно выбранная архитектура обеспечивает масштабируемость приложения, упрощает его тестирование и поддержку, повышает читаемость кода. В контексте Android-разработки архитектурные паттерны играют критическую роль, поскольку мобильные приложения характеризуются сложным жизненным циклом компонентов и необходимостью постоянного управления состоянием пользовательского интерфейса.

В мобильной разработке применяются несколько основных архитектурных паттернов, каждый из которых имеет свои особенности и области применения.

MVC (Model-View-Controller) представляет собой классический паттерн проектирования, который разделяет приложение на три компонента. Model отвечает за данные и бизнес-логику приложения, View представляет пользовательский интерфейс и отображает данные пользователю, Controller обрабатывает действия пользователя и координирует взаимодействие между Model и View. Основная проблема такого подхода заключается в том, что эти компоненты быстро становятся перегруженными кодом. Они одновременно управляют интерфейсом, обрабатывают действия пользователя и содержат бизнес-логику, что затрудняет тестирование и поддержку приложения. MVC использовался на ранних этапах развития Android, однако со временем

выявились его недостатки для мобильной разработки. Сегодня этот паттерн считается устаревшим для современных Android-приложений.

MVP (Model-View-Presenter) является улучшенной версией MVC. В этом паттерне между Model и View находится Presenter, который берет на себя всю логику представления данных. View становится пассивным компонентом, который только отображает данные, предоставленные Presenter, и передает ему действия пользователя. View, в свою очередь, не содержит никакой логики и не обращается напрямую к Model. Главное преимущество MVP состоит в том, что Presenter не зависит от Android-компонентов. Это обычный класс на языке программирования, который легко тестировать без необходимости запуска Android-окружения. Однако у паттерна есть существенный недостаток. Для связи View и Presenter приходится создавать множество интерфейсов, что значительно увеличивает объем кода. Несмотря на это, MVP был популярен в Android-разработке в период до появления современных инструментов от Google.

MVVM (Model-View-ViewModel) представляет современный подход к построению архитектуры Android-приложений. Паттерн также состоит из трех компонентов. Model отвечает за данные и бизнес-логику, View представляет пользовательский интерфейс, а ViewModel выступает связующим звеном между ними, предоставляя данные для отображения и обрабатывая действия пользователя. Ключевое отличие MVVM от предыдущих паттернов заключается в использовании механизма привязки данных. View автоматически обновляется при изменении данных в ViewModel. Разработчику не нужно вручную обновлять элементы интерфейса – система делает это сама при изменении данных. В Android это реализуется через LiveData

или StateFlow, которые обеспечивают реактивное обновление интерфейса. Еще одно важное преимущество ViewModel – она сохраняет данные при изменениях конфигурации устройства. Например, при повороте экрана Android пересоздает Activity, но ViewModel остается в памяти. Это решает проблему потери состояния приложения, которая была характерна для MVC и MVP.

Параметр	MVC	MVP	MVVM
Сложность реализации	Низкая	Средняя	Средняя
Тестируемость	Низкая	Высокая	Высокая
Объем кода	Небольшой	Большой	Средний
Разделение ответственности	Слабое	Хорошее	Хорошее
Зависимость от платформы	Высокая	Средняя	Низкая
Поддержка реактивности	Нет	Частичная	Полная
Актуальность для Android	Устаревший	Частично	Рекомендуется

Таблица 4. Сравнительная характеристика архитектурных паттернов

Компания Google предоставляет официальные рекомендации по архитектуре Android-приложений, которые основываются на принципах MVVM. Рекомендуемая структура приложения состоит из нескольких слоев с четким разделением ответственности. UI Layer отвечает за отображение данных и обработку взаимодействия с пользователем. Domain Layer содержит бизнес-логику приложения, представленную в виде Use Cases —

отдельных операций, которые может выполнить система. Data Layer управляет источниками данных и обеспечивает их предоставление вышележащим слоям. Такая организация позволяет независимо развивать каждый слой и упрощает замену отдельных компонентов без влияния на остальную систему.

Для реализации архитектуры Google предоставляет набор библиотек Jetpack Architecture Components. ViewModel представляет собой компонент для хранения и управления данными интерфейса с учетом жизненного цикла Android-компонентов. ViewModel сохраняет состояние при изменениях конфигурации устройства, что избавляет от необходимости повторной загрузки данных. StateFlow представляет собой поток данных, который содержит текущее значение и уведомляет подписчиков об изменениях. StateFlow обеспечивает простую интеграцию с Jetpack Compose и корутинами Kotlin. Room представляет собой библиотеку для работы с базой данных SQLite, которая предоставляет уровень абстракции над SQL-запросами и проверяет их корректность на этапе компиляции.

Jetpack Compose представляет собой декларативный фреймворк для создания пользовательского интерфейса. В отличие от традиционного подхода с XML-разметкой, Compose позволяет описывать интерфейс в коде Kotlin. Compose-функции помечаются аннотацией Composable и автоматически обновляются при изменении данных. Это обеспечивает естественную интеграцию с паттерном MVVM и StateFlow.

Для управления зависимостями применяется библиотека Hilt — надстройка над Dagger 2, оптимизированная для Android. Hilt автоматизирует создание и управление зависимостями через систему аннотаций. ViewModel помечаются аннотацией HiltViewModel, а зависимости внедряются через

конструктор с аннотацией Inject. Модули Hilt определяют способы создания сложных объектов. Такой подход упрощает архитектуру и обеспечивает корректное управление жизненным циклом объектов.

Для данного проекта была выбрана архитектура MVVM с разделением на слои. Presentation Layer включает Jetpack Compose компоненты для построения интерфейса и ViewModel для управления состоянием экранов. Domain Layer содержит Use Cases, которые инкапсулируют бизнес-логику операций, таких как получение списка мероприятий или регистрация пользователя. Data Layer включает Repository для абстракции источников данных и Data Sources для работы с локальными и удаленными источниками. Выбор данной архитектуры обусловлен необходимостью четкого разделения ответственности, обеспечения реактивного обновления интерфейса и соответствия современным стандартам Android-разработки, рекомендованным компанией Google.

1.4. Методы создания UI и работы с данными в Android

1.4. Методы создания UI и работы с данными в Android

Создание пользовательского интерфейса и организация работы с данными представляют собой ключевые аспекты разработки Android-приложений. В данном разделе рассматриваются современные подходы к построению UI и методы управления данными, применяемые в мобильных приложениях.

Традиционно пользовательский интерфейс в Android создавался с использованием XML-разметки. Разработчик описывал структуру экрана в XML-файлах, определяя расположение элементов, их размеры и свойства. Данный подход обеспечивал разделение логики приложения и визуального представления, позволял использовать визуальный редактор для проектирования интерфейса и поддерживал адаптацию под различные размеры экранов. Однако XML-подход имел существенные недостатки: большой объем шаблонного кода, сложность динамического изменения интерфейса, необходимость связывания XML-элементов с кодом и ограниченные возможности переиспользования компонентов.

Jetpack Compose представляет собой современный декларативный фреймворк для создания пользовательского интерфейса Android-приложений. В отличие от императивного подхода с XML, где разработчик явно описывает последовательность действий для построения UI, Compose использует декларативную модель программирования. Разработчик описывает, как должен выглядеть интерфейс в зависимости от текущего состояния данных, а фреймворк автоматически обновляет отображение при их изменении. Compose-функции являются обычными Kotlin-функциями, помеченными

аннотацией Composable, что обеспечивает полную интеграцию с языком и его возможностями.

Преимущества Jetpack Compose включают значительное сокращение объема кода за счет устранения XML-файлов, улучшенную производительность благодаря интеллектуальной перерисовке только измененных компонентов, упрощенное создание анимаций и встроенную поддержку Material Design 3. Compose также обеспечивает лучшую типобезопасность, поскольку вся разметка пишется на Kotlin, что позволяет обнаруживать ошибки на этапе компиляции. В рамках данного проекта Jetpack Compose применяется для создания всех экранов приложения: главной страницы, списка мероприятий, детальной информации о событиях и страницы авторизации.

Material Design представляет собой систему дизайна, разработанную Google, которая определяет принципы создания пользовательских интерфейсов. Текущая версия Material Design 3 предоставляет адаптивные цветовые схемы, улучшенную типографику и обновленные компоненты. Jetpack Compose включает полную поддержку Material Design 3 через библиотеку Material3, предоставляющую готовые компоненты, соответствующие стандартам дизайна. В приложении используются компоненты Material3, такие как TopAppBar для навигационной панели, Card для отображения карточек мероприятий, Button для интерактивных элементов и TextField для ввода данных.

Для локального хранения данных в Android существуют различные подходы. DataStore представляет собой современное решение для хранения пар ключ-значение, пришедшее на смену SharedPreferences. DataStore

обеспечивает типобезопасность, поддержку корутин для асинхронных операций и защиту от ошибок при одновременном доступе. В данном проекте DataStore применяется для хранения информации об авторизованном пользователе, что позволяет сохранять сессию между запусками приложения.

Room представляет собой библиотеку из состава Android Jetpack, предоставляющую уровень абстракции над SQLite базой данных. Room упрощает работу с базой данных, автоматически генерируя реализацию методов доступа к данным на основе аннотированных интерфейсов и проверяя SQL-запросы на этапе компиляции. Room состоит из трех основных компонентов: Entity (сущности, представляющие таблицы базы данных), DAO (Data Access Object, определяющие методы доступа к данным) и Database (класс, предоставляющий точку входа в базу данных). Хотя в текущей версии приложения Room не используется, архитектура предусматривает возможность его интеграции для кеширования данных о мероприятиях.

Для взаимодействия с серверной частью применяются библиотеки для выполнения HTTP-запросов. Retrofit представляет собой типобезопасный HTTP-клиент для Android, который преобразует HTTP API в Kotlin-интерфейсы. Retrofit автоматически сериализует и десериализует данные, поддерживает различные форматы и интегрируется с корутинами для асинхронного выполнения запросов. В перспективе Retrofit будет использован для получения списка мероприятий с сервера, отправки регистраций пользователей и синхронизации данных.

Для сериализации и десериализации JSON-данных применяется `kotlinx.serialization` — официальная библиотека сериализации для Kotlin. В отличие от рефлексивных библиотек, `kotlinx.serialization` использует генерацию

кода на этапе компиляции, что обеспечивает типобезопасность и высокую производительность. В приложении `kotlinx.serialization` используется для сериализации объектов `User` при сохранении в `DataStore`, что позволяет надежно хранить структурированные данные.

Асинхронная обработка данных является критически важной для мобильных приложений. Kotlin Coroutines представляют собой механизм для написания асинхронного кода в последовательном стиле. Корутины позволяют приостанавливать выполнение функции без блокировки потока, автоматически переключаться между потоками и обрабатывать ошибки. Suspend-функции, помеченные ключевым словом `suspend`, могут выполнять длительные операции, такие как загрузка данных или работа с базой данных, не блокируя главный поток интерфейса. В приложении корутины используются в `ViewModel` для загрузки списка мероприятий и работы с `DataStore`.

Kotlin Flow представляет собой API для работы с асинхронными потоками данных. Flow позволяет создавать последовательности значений, которые генерируются асинхронно, применять операторы трансформации и обрабатывать ошибки. `StateFlow` является специализированным типом `Flow`, который всегда имеет текущее значение и уведомляет подписчиков об его изменении. `StateFlow` идеально подходит для управления состоянием пользовательского интерфейса в паттерне MVVM. В проекте `StateFlow` используется во всех `ViewModel` для хранения состояния экранов: списка мероприятий, информации о текущем пользователе и деталей выбранного события. Composable-функции подписываются на `StateFlow` через `collectAsState()`, автоматически перерисовываясь при изменении данных.

Архитектура работы с данными в приложении организована по принципам Clean Architecture. LocalDataSource предоставляет локальные данные о мероприятиях, которые в будущем будут заменены на запросы к серверу. EventRepository абстрагирует источник данных, предоставляя единый интерфейс для получения мероприятий. AuthRepository управляет аутентификацией пользователя, сохраняя и загружая данные из DataStore. Use Cases инкапсулируют бизнес-логику отдельных операций, таких как GetEventsUseCase для получения списка мероприятий. ViewModel использует Use Cases для получения данных и управляет состоянием экранов через StateFlow.

Таким образом, выбранный набор технологий обеспечивает создание современного и производительного приложения. Jetpack Compose упрощает разработку и поддержку пользовательского интерфейса, обеспечивая декларативный подход к построению UI. Корутины и Flow предоставляют эффективные средства для асинхронной обработки данных и управления состоянием. DataStore обеспечивает надежное локальное хранение данных пользователя. kotlinx.serialization гарантирует типобезопасную работу с JSON. Все выбранные технологии соответствуют современным стандартам Android-разработки, имеют официальную поддержку Google и широко применяются в индустрии.

2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

2.1. Анализ требований к системе

2.2. Архитектура системы и моделирование компонентов

2.3. Моделирование данных на мобильной стороне

2.4. Проектирование рекомендательной системы

3. РАЗРАБОТКА МОБИЛЬНОЙ-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

3.1. Разработка доменного слоя

3.2. Взаимодействие с backend-частью

3.3. Проектирование пользовательского интерфейса

3.4. Интеграция рекомендательной системы

3.5. Тестирование и оптимизация приложения

ЗАКЛЮЧЕНИЕ

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Кашапов М.Н. Разработка дизайна приложения, Казань, 2019. – 43 с. [Электронный ресурс]. URL: https://kpfu.ru/student_diplom/10.160.178.20_2627777_Kashapov_M.N._razrabotka_dizajna_prilozheniya.pdf (дата обращения: 09.02.2026).
2. Петросян Л. Э., Гусев К.В. Разработка мобильных приложений на языке Kotlin, Учебное пособие для вузов. Издательство "Лань". 2025. – 104 с. [Электронный ресурс]. URL: <https://e.lanbook.com/book/448577> (дата обращения: 09.02.2026).
3. Эгов Е.Н., Кянжин С.О. Проектирование мобильного приложения для IT-компаний для бронирования переговорных на платформе Android с использованием языка Kotlin, 2023. – 176 с. [Электронный ресурс]. URL: <https://elibrary.ru/item.asp?id=54328731> (дата обращения: 09.02.2026).
4. Степанов П. В., Приходько Н. А., Запорожских А. И. Разработка мобильных приложений под Android на языке Kotlin, учебное пособие. МИРЭА - Российский технологический университет. 2025. – 889 с. [Электронный ресурс]. URL: <https://e.lanbook.com/book/504855> (дата обращения: 10.02.2026).
5. Орлов А.В. Современная разработка Android-приложений и язык программирования Kotlin. 2023. – 73 с. [Электронный ресурс]. URL: <https://www.elibrary.ru/item.asp?id=50177358> (дата обращения: 10.02.2026).
6. Грузин Н.А. Сравнение шаблонов проектирования архитектуры приложения MVC, MVP и MVVM. [Электронный ресурс]. URL: <https://www.elibrary.ru/item.asp?id=44555466> (дата обращения: 10.02.2026).

7. Артеменко А.В., Бабушкина Н.Е. Ролевая архитектура web-приложений MVC, MVVM и их влияние на код. [Электронный ресурс]. URL: <https://www.elibrary.ru/item.asp?id=60274200> (дата обращения: 10.02.2026).
8. Турушев Т.К. Архитектурные паттерны проектирования системы MVC, MVP, MVVM. [Электронный ресурс]. URL: <https://www.elibrary.ru/item.asp?id=47149990> (дата обращения: 10.02.2026).
9. К вопросу выбора оптимального языка программирования для разработки мобильного приложения. [Электронный ресурс]. URL: https://elibrary.ru/download/elibrary_50024700_66439187.pdf (дата обращения: 10.02.2026).
10. Приходько Н.А., Сивченко В.А. Причины выбора Kotlin как основного языка мобильной разработки на Android. [Электронный ресурс]. URL: <https://www.elibrary.ru/item.asp?id=56114037> (дата обращения: 10.02.2026).
11. Мобильные приложения и их роль в современном мире. Типы мобильных приложений. Операционные системы мобильных устройств [Электронный ресурс]. URL: <https://science.kuzstu.ru/wp-content/Events/Conference/RM/2021/RM21/pages/Articles/031434.pdf> (дата обращения: 05.02.2026).
12. Digital 2025, Global Overview Report. [Электронный ресурс]. URL: <https://www.vennapps.com/blog/mobile-app-download-and-usage-statistics-2025> (дата обращения: 05.02.2026).
13. Kotlin docs. [Электронный ресурс]. URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 26.01.2026).
14. Документация React Native. [Электронный ресурс]. URL: <https://reactnative.dev/docs/getting-started> (дата обращения: 27.01.2026).

15. Документация Flutter. [Электронный ресурс]. URL: <https://flutter.dev/> (дата обращения: 27.01.2026).
16. Документация Android. [Электронный ресурс]. URL: <https://developer.android.com/get-started/overview> (дата обращения: 27.01.2026).
17. Документация Material Design. [Электронный ресурс]. URL: <https://m3.material.io/> (дата обращения: 27.01.2026).
18. Документация Hilt. [Электронный ресурс]. URL: <https://dagger.dev/hilt/> (дата обращения: 27.01.2026).
19. Методология Agile. [Электронный ресурс]. URL: <https://practicum.yandex.ru/blog/metodology-agile/> (дата обращения: 27.01.2026).
20. Рекомендательные системы. [Электронный ресурс]. URL: <https://simulative.ru/blog/rekomendatelnie-sistemi-kak-ml-inzheneri-sozdayut> (дата обращения: 27.01.2026).
21. Рекомендательная система через TF-IDF. [Электронный ресурс]. URL: <https://habr.com/ru/companies/otus/articles/755772/> (дата обращения: 27.01.2026).
22. Микросервисная архитектура. [Электронный ресурс]. URL: <https://habr.com/ru/companies/raiffeisenbank/articles/346380/> (дата обращения: 27.01.2026).
23. Сама функция TF-IDF в Sklearn. [Электронный ресурс]. URL: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (дата обращения: 27.01.2026).
24. FastAPI. [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.01.2026).
25. Россия молодая: сборник материалов XIII Всероссийской научно-практической конференции молодых ученых с международным участием

[Электронный ресурс]. URL: https://www.elibrary.ru/download/elibrary_47123249_74986550.pdf (дата обращения: 11.02.2026).

26. Первун О.Е., Скрипка В.К. Операционная система Android: история и перспективы [Электронный ресурс]. URL: <https://elibrary.ru/item.asp?id=36605581> (дата обращения: 12.02.2026).

ПРИЛОЖЕНИЕ

Репозиторий – <https://github.com/Shtskkh/Events.Android>